

Apuntes y Ejercicios

Martín Mateos

28 de agosto de 2026

Índice

1. Guía de ayuda para escritura Latex	2
2. Repaso de demostraciones	2
2.1. Guía práctica 1	2
3. Introducción a grafos	2
3.1. Guía práctica 2	2
3.2. Algoritmos sobre grafos	2
3.2.1. Representación	2
3.2.2. Ordenamiento topológico	3
3.2.3. Recorridos sobre grafos - BFS	3
3.2.4. Recorridos sobre grafos - DFS	3
3.2.5. Aristas de corte	3
3.2.6. Guía práctica 3	3

1. Guía de ayuda para escritura Latex

Una fórmula matemática en línea se escribe entre signos de pesos: $E = mc^2$.

Para centrar una ecuación en un bloque independiente:

$$\int_a^b f(x) dx = F(b) - F(a)$$

Ejercicio 1. Calcule la derivada de la función $f(x) = x^3 + 2x^2 - 5$.

Ejercicio 2. Demuestre la siguiente identidad utilizando las propiedades fundamentales:

$$\sin^2(\theta) + \cos^2(\theta) = 1 \tag{1}$$

2. Repaso de demostraciones

2.1. Guía práctica 1

3. Introducción a grafos

3.1. Guía práctica 2

hola

3.2. Algoritmos sobre grafos

Esta sección define el ordenamiento topológico para luego hacer énfasis encómo recorrer un grafo mediante algoritmos, los costos operacionales de estos, las demostraciones formales de los algoritmos y la información obtenida luego de sus implementaciones. Finalmente, se verá qué es una arista de corte y cual es su importancia.

3.2.1. Representación

Como objeto matemático, un grafo es un par ordenado $G = (V, E) : v, w \in V, (v, w) \in E$. Como estructura de datos, un grafo puede representarse mediante una lista enlazada o una matriz de dimensión $n \times n$ con $n = |V(G)|$.

Cada estructura debe admitir las siguientes operaciones:

- Consultar si $(v, w) \in E$.
- Recorrer $N(v)$.
- Insertar o remover aristas.
- Insertar o remover vértices.

Donde $\text{adj}[v] = N(v) = \{w \in V : (v, w) \in E\}$, es decir el conjunto de vecinos de v .

Una implementacion del grafo como lista enlazada de n nodos $V = \{v_1, v_2, \dots, v_n\}$ sobre listas sería:

```

Nodo{
    id: Insertar
    ady: List<Nodo>
}

Grafo{
    nodos: List<Nodo>
}

```

Entonces cada nodo en la lista hace referencia a sus vecinos, mientras que una implementación del grafo por una matriz es simplemente una lista de listas donde el elemento ij representa que si existe (v_i, v_j) . A continuación de las complejidades de cada estructura, con $n = V(D) \wedge m = A(D)$:

Operación	Lista Enlazada	Matriz
Espacio	$\Theta(n + m)$	$\Theta(n^2)$
Consultar $(v, w) \in E(D)$	$O(d(v))$	$O(n^2)$
Recorrer $N(v)$	$O(d(v))$	$O(n)$
Recorrer todas las aristas	$O(n + m)$	$O(n^2)$

Si el grafo posee pesos, estos se almacenan junto con sus vecinos. Para listas $\text{adj}(v) = \langle (w_1, p_1), \dots, (w_k, p_k) \rangle$ y para la matriz simplemente el valor del peso en el elemento ij teniendo en cuenta que $D[i][j] = \text{null}$ indica la ausencia de adyacencia.

3.2.2. Ordenamiento topológico

Sea $D = (V, E)$ un digrafo. Un **ordenamiento topológico** de D es un orden lineal $v_1, v_2, \dots, v_n \in V(D)$ tal que $v_i \rightarrow v_j \in V(D) \Rightarrow i < j$.

Lema:

Todo digrafo acíclico (DAG) tiene un vértice v tal que $d(v) = 0$.

Teorema:

Un digrafo admite ordenamiento topológico si y sólo si es acíclico.

Demostraciones en apuntes.

3.2.3. Recorridos sobre grafos - BFS

3.2.4. Recorridos sobre grafos - DFS

3.2.5. Aristas de corte

3.2.6. Guía práctica 3

Ejercicio 1. Para un grafo G el **conjunto de vencidarios** o conjunto de adyacencias es el par $(V(G), N)$ donde N es la función definida en la sección 3.2.1 que devuelve el conjunto de vecinos de un vector $v \in V(G)$.

Suponer que los vértices dados son el conjunto $\{0, 1, \dots, |V(G)| - 1\}$.

Discutir las ventajas y desventajas en cuanto a la complejidad temporal y espacial de las siguientes implementaciones del conjunto de vencidarios del grafo G , de acuerdo a las siguientes operaciones:

1. Inicializar la estructura a partir de una secuencia de vértices y una secuencia de aristas de G .
2. Determinar si dos vértices v y w son adyacentes.
3. Recorrer y/o procesar el vecindario $N(v)$ de un vértice v dado.
4. Insertar un vértice v con su conjunto de vecinos $N(v)$.
5. Insertar una arista (v, w) .
6. Remover un vértice v con todas sus adyacencias.
7. Remover una arista (v, w) .
8. Mantener un orden de $N(v)$ de acuerdo a algún invariante que permita recorrer cada vecindario en un orden dado.

Estructuras de datos:

1. La función N se representa con una secuencia (arreglo dinámico o lista enlazada) que en cada posición v tiene el conjunto $N(v)$ implementado a su vez sobre una secuencia (arreglo dinámico o lista enlazada). Cada vértice es una estructura que tiene un índice para acceder en $O(1)$ a $N(v)$. Esta representación se conoce comúnmente como lista de adyacencias.
2. Ídem anterior, pero cada $w \in N(v)$ se almacena junto con un índice a la posición que ocupa v en $N(w)$. Esta representación también se conoce como lista de adyacencias, pero tiene información para implementar operaciones dinámicas.
3. $N(v)$ se representa con un arreglo dinámico que en cada posición i tiene un arreglo dinámico de booleanos A_i con $|V(G)|$ posiciones tal que $A_i[j]$ es verdadero si y solo si i es adyacente a j . Esta representación se conoce comúnmente como matriz de adyacencias.
4. $N(v)$ se representa con un arreglo dinámico que en cada posición tiene el conjunto $N(v)$ implementado con una tabla de hash. Esta representación es un mix entre las representaciones clásicas de matriz de adyacencias y lista de adyacencias.

Respuestas:

Voy a definir el largo de la secuencia de vectores con n y el largo de la secuencia de aristas con m .

1.1)

```
Es es List<List<Int>>>
Vs es List<Int>
```

```
inicializar(V:Vs, es:Es): list<list<int>>>{
    for(i=0, i++, V.len()){
        res.append([])
    }
    forall e in es{ // recorro solo n (las aristas) 0(len(n))
```

```

        res[es[0]].append(es[1])
        res[es[1]].append(es[0])
    }
}

```

Para arreglos, encolamos al final n listas vacías ($O(n)$), luego recorremos la vecindad obteniendo las aristas (v, w) tal que `res[v].append(w)` en cada iteración. Esto es $O(m)$.

En el caso de las listas enlazadas habría que recorrer la lista de vectores ($O(n)$), luego ordenarla para que el número de la posición coincida con los vectores ($O(n^2)$). Nos movemos a la secuencia de aristas y en la posición v de `res` colocamos w y viceversa, acceder a un elemento en la lista enlazada es $O(n)$ y como lo hacemos m veces tenemos que la complejidad de almacenar una arista es $O(n \cdot m)$.

Finalmente la complejidad total es $O(n^2 + n \cdot m)$

En cuanto al espacio utilizado, los arrays toman $O(n + m)$ y las listas enlazadas $O(n + m)$, así que por lo tanto, desempataando por la complejidad temporal me quedo con los arreglos.

1.2 En listas enlazadas buscar a v es $O(n)$ y buscar a w es $O(d(v))$, por lo tanto la complejidad de determinar si $w \in N(v)$ es $O(n + d(v))$.

En arreglos dinámicos buscar a v y w es $O(1)$

Ejercicio 2. -Enunciado-

Respuestas para el algoritmo cúbico: Para ver mejor el algoritmo intento implementarlo en pseudocódigo, busco vectores conectados entre sí tal que si A es la matriz de adyacencias entonces $A_{vw} A_{wz} A_{wv} = 1$.

```

for(v=0, v++, A.lenght){ // n op
    for(w=1, v++, A.lenght){ // n-1 ops
        for(z=2, v++, A.lenght){ // n-2 ops
            if (A[v][w]*A[w][z]*A[z][w] == 1){ // 1 ops
                return true; //1 ops
            }
        }
    }
}

```

a) El algoritmo es correcto porque explora todas las combinaciones posibles de triángulos.

b) Para responder apropiadamente debo ver el numero de operaciones.

$$\begin{aligned}
 T(n) &= \sum_{v=0}^n \sum_{w=2}^n \sum_{z=3}^n 2 \\
 &= 2 \left(\sum_{v=1}^n \sum_{w=1}^{n-1} \sum_{z=1}^{n-2} 1 \right) \\
 &= 2(n-2) \left(\sum_{v=1}^n \sum_{w=1}^{n-1} 1 \right) \\
 &= (n-2)(n-1) \sum_{v=1}^n 1 \\
 &= (n-2)(n-1)n
 \end{aligned}$$

Como $T(n) \approx n^3 \wedge n^3 \in O(n^3) \wedge n^3 \in \Omega(n^3) \implies T(n) \in \Theta(n^3)$ (habría que demostrar los dos pertenece anterior).

c) El peor caso ya lo mencioné, cuando el triangulo son los últimos 3 nodos. El mejor caso serían los tres primeros nodos.

Respuestas algoritmo cuadrado: Si la lista de adyacencias es `L:list<list<int>>`, iniciarla es $O(n + m)$:

```

for(v=0, v++, L.lenght){
    visitados = []
    for(j=0, w++, L[v].lenght){ // 0(n)
        if(pertenece(L[v][j], visitados))
    }
}

```